ABSTRACT

ABSTRACT

Communication is an important part of human life, but people with hearing and speech impairments often face difficulties while interacting with others. Sign language is the primary mode of communication for such individuals, but most people are not familiar with sign language, creating a communication gap. To overcome this issue, the **Real-Time Sign Language Text-to-Speech Translator** is developed as an intelligent and user-friendly system that converts sign language gestures into text and speech in real time.The main objective of this project is to recognize hand gestures using computer vision and machine learning techniques and provide instant text and voice output. The system captures live video input through a webcam and processes the frames using image processing techniques. Hand landmarks and finger movements are detected using MediaPipe and OpenCV libraries. The detected gestures are then analyzed and classified using machine learning algorithms trained with predefined gesture datasets.Once a gesture is recognized, the corresponding text is displayed on the screen and converted into speech using Text-to-Speech (TTS) technology. This enables effective communication between hearing and speech-impaired individuals and normal users without the need for a human interpreter.The system is designed to work in real time with high accuracy and minimal delay. It provides a simple and user-friendly interface, making it easy to use for people of all age groups. The project demonstrates the practical implementation of Artificial Intelligence, Machine Learning, and Speech Processing technologies in solving real-world communication problems.The Real-Time Sign Language Text-to-Speech Translator helps improve accessibility, promotes independence, and enhances social inclusion for people with disabilities. Future enhancements may include support for additional gestures, multiple languages, and mobile application integration for wider usability.

CONTENT

CONTENT

INTRODUCTION

1. INTRODUCTION

Communication is an essential part of human interaction, enabling people to share ideas, emotions, and information. However, individuals with hearing and speech impairments face significant difficulties in communicating with others. They mainly rely on sign language, which uses hand gestures, facial expressions, and body movements. Since most people are not familiar with sign language, a communication gap exists between deaf or mute individuals and the general public.

To overcome this challenge, the Real-Time Sign Language Text-to-Speech Translator is developed. This system is designed to recognize hand gestures and convert them into readable text and audible speech in real time. The solution makes use of modern technologies such as Artificial Intelligence, Machine Learning, and Computer Vision to detect and interpret sign language gestures captured through a camera.

The system works by capturing live video input, processing the hand gestures using trained models, and mapping them to corresponding text meanings. The recognized text is then converted into speech using a text-to-speech mechanism. This enables smooth and effective communication between sign language users and others without requiring an interpreter.

The main objective of this project is to reduce communication barriers and provide an accessible, efficient, and user-friendly solution for people with hearing and speech disabilities. The system helps in improving interaction, promoting independence, and increasing social inclusion. In conclusion, the Real-Time Sign Language Text-to-Speech Translator demonstrates how technology can be used to solve real-world communication problems and create a more inclusive environment for everyo

SYSTEM STUDY

2. SYSTEM STUDY

2.1.EXISTING SYSTEM

In the existing system, communication for hearing and speech-impaired individuals mainly depends on sign language. However, most people are not trained to understand sign language, which creates a major communication barrier.

Currently, the following methods are used:

- Human interpreters are required to translate sign language into spoken language
- Written communication is used in some cases
- Basic gesture-based applications exist but are limited in functionality

Limitations of Existing System:

- Requires the presence of a trained interpreter
- Not suitable for real-time communication in all situations
- Time-consuming and less efficient
- Limited accessibility for common people
- Existing applications have low accuracy and limited gesture recognition

These drawbacks highlight the need for an automated system that can perform real-time translation without human assistance.

2.2.PROPOSED SYSTEM

The proposed system is a Real-Time Sign Language Text-to-Speech Translator that uses Artificial Intelligence and Computer Vision techniques to recognize hand gestures and convert them into text and speech instantly.

The system captures live video input through a camera and processes the hand gestures using trained machine learning models. The recognized gesture is converted into text, which is then transformed into speech using a text-to-speech engine.

Features of Proposed System:

- Real-time gesture recognition
- Automatic conversion of sign language to text
- Instant speech output
- User-friendly interface
- No need for human interpreter

Advantages of Proposed System:

- Faster and more efficient communication
- Reduces dependency on others
- Easy to use for both normal users and disabled individuals
- Improves accessibility and inclusivity
- Can be used in various environments like schools, hospitals, and public places

SYSTEM CONFIGURATION

1. SYSTEM CONFIGURATION

1.1 HARDWARE CONFIGURATION

Component	Specification
Processor	Intel Core i3 / i5 or higher
RAM	Minimum 4GB (8GB recommended for better performance)
Storage	500GB HDD or SSD
Camera	Webcam or inbuilt laptop camera (for capturing hand gestures)

1.2 SOFTWARE CONFIGURATION

Software Component	Specification / Version Used
Operating System	Windows 10 / Windows 11
Programming Language	Python
Libraries and Tools Used	OpenCV (for image processing) MediaPipe (for hand tracking and gesture detection) TensorFlow / Machine Learning libraries (for model training) Text-to-Speech Engine (such as pyttsx3 or gTTS)
Development Environment	VS Code / PyCharm / Jupyter Notebook

1.3 SOFTWARE DESCRIPTION

The Real-Time Sign Language Text-to-Speech Translator is developed using the Python programming language and integrates various advanced technologies such as Computer Vision, Machine Learning, and Speech Processing to achieve accurate and real-time communication. The software is designed to capture hand gestures through a camera, interpret them using intelligent algorithms, and convert them into meaningful text and speech output.

Overview of the System

The software works as an interactive system that continuously processes live video input. It detects hand gestures, interprets them into predefined sign language meanings, and provides output in both text and audio formats. The entire process happens in real time, ensuring smooth and natural communication.

The system architecture is divided into multiple functional modules such as:

- Video Capture Module
- Image Processing Module
- Hand Detection and Tracking Module
- Gesture Recognition Module
- Text Conversion Module
- Speech Generation Module

Each module plays a vital role in ensuring the overall efficiency and accuracy of the system.

Video Capture and Image Processing

The first step in the system is capturing real-time video input using a webcam. This is achieved using the OpenCV library, which provides tools for video capturing and frame processing. The video is divided into individual frames, and each frame is analyzed separately.

Image preprocessing techniques are applied to improve detection accuracy. These include:

- Resizing frames for faster processing
- Converting color formats (RGB/BGR)
- Removing noise from the image
- Adjusting brightness and contrast

These preprocessing steps ensure that the system performs consistently under different lighting and background conditions.

Hand Detection and Tracking

After preprocessing, the system identifies the presence of a hand in the video frame. MediaPipe is used for this purpose, as it provides a highly efficient and accurate hand tracking solution. It detects multiple hand landmarks, such as finger tips, joints, and palm positions.

MediaPipe identifies around 21 key points on the hand, which are used to understand the structure and movement of the hand. These landmarks are tracked continuously across frames, allowing the system to recognize dynamic gestures as well as static ones.

This module ensures:

- Accurate detection of hand position
- Tracking of finger movements
- Real-time performance with minimal delay

Gesture Recognition using Machine Learning

The detected hand landmarks are passed to a Machine Learning model, which is responsible for recognizing the gesture. The model is trained using a dataset containing various sign language gestures. Each gesture is associated with a specific label such as alphabets (A–Z), numbers, or common words.

The recognition process involves:

- Feature extraction from hand landmarks
- Classification using trained algorithms
- Matching input gestures with stored patterns

The machine learning model improves the system's ability to handle variations in hand size, orientation, and movement. This increases the accuracy and robustness of the system.

Text Conversion Module

Once a gesture is recognized, the corresponding meaning is converted into text format. The system maintains a mapping between gestures and their respective text outputs.

For example:

- Gesture → "Hello" → Displayed as text on screen
- Gesture → "Thank You" → Displayed as text

The text output is shown in real time on the user interface, allowing users to visually confirm the recognized gesture.

Speech Generation Module

The text generated from the gesture recognition module is then passed to a Text-to-Speech (TTS) engine. This module converts the text into audible speech using voice synthesis techniques.

Common TTS tools used include:

- pyttsx3 (offline speech synthesis)
- gTTS (Google Text-to-Speech)

This feature is very important as it enables users to communicate verbally with others. The speech output is clear, natural, and generated instantly after gesture recognition.

Real-Time Processing and Performance

One of the key features of the system is real-time processing. The software continuously captures video input, processes frames, detects gestures, and generates output without noticeable delay.

To achieve this, the system is optimized for:

- Fast frame processing
- Efficient memory usage
- Low latency output

The system ensures smooth interaction and quick response, which is essential for practical communication.

User Interface and Usability

The software is designed with a simple and user-friendly interface. Users can easily start the camera, perform gestures, and view the output without any technical knowledge.

Key usability features include:

- Easy camera access
- Real-time text display
- Instant audio feedback
- Minimal user interaction required

The interface ensures accessibility for both technical and non-technical users.

System Efficiency and Reliability

The software is built to be efficient, reliable, and scalable. It performs well under different environmental conditions and maintains consistent accuracy.

Key performance aspects:

- High gesture recognition accuracy
- Fast processing speed
- Stable performance over long usage
- Ability to handle multiple gestures

The integration of Computer Vision and Machine Learning ensures that the system adapts well to real-world scenarios.

SYSTEM DESIGN AND DEVELOPMENT

2. SYSTEM DESIGN AND DEVELOPMENT

The System Design and Development phase is a critical stage in the software development life cycle. It involves transforming the system requirements into a structured design and then implementing it into a working system. In this project, the system is designed to recognize sign language gestures and convert them into text and speech output in real time. The design focuses on creating an efficient, accurate, and user-friendly system that can be easily used by both hearing-impaired individuals and general users. The development process ensures smooth interaction between different modules and fast processing of real-time data.

4.1 SYSTEM ARCHITECTURE

The system follows a modular architecture where each module performs a specific function. This modular design improves flexibility, scalability, and ease of maintenance.

Main Components of the System:

- Camera Input Module
- Image Processing Module
- Hand Detection Module
- Gesture Recognition Module
- Text Conversion Module
- Speech Output Module
- User Interface Module

Working Architecture Flow:

1. The camera captures real-time video input
2. Video is divided into frames
3. Frames are processed using image processing techniques
4. Hand is detected and tracked
5. Gesture is recognized using a trained model
6. Recognized gesture is converted into text
7. Text is converted into speech
8. Output is displayed and played

This pipeline ensures continuous and real-time operation of the system.

4.2 INPUT DESIGN

Input design is responsible for capturing and handling user input effectively.

Input Type:

- Live video stream
- Hand gestures representing sign language

Input Design Features:

- Continuous frame capture
- Automatic hand detection
- Support for different gesture styles
- Works under various lighting conditions

Input Requirements:

- Proper camera positioning
- Clear background for better detection
- Visible hand gestures within frame

The system is designed so that users can give input naturally without any special training.

4.3 OUTPUT DESIGN

Output design determines how the system presents results to the user.

Types of Output:

1. Text Output
    - Displays recognized gesture as text
    - Helps user verify correctness
2. Audio Output
    - Converts text into speech
    - Enables communication with others

Output Features:

- Real-time response
- Clear text display
- Natural voice output
- Minimal delay

Example**:**

Gesture → “Hello” → Text displayed → Audio played

The output system ensures effective communication and user satisfaction.

4.4 DATABASE DESIGN

The database stores gesture-related data and mappings required for recognition.

Database Contents:

- Gesture ID
- Gesture Data (images / landmarks)
- Corresponding Meaning (text)

Functions of Database:

- Store training dataset
- Map gestures to text
- Support machine learning model

The database is designed to allow fast access and efficient data handling.

4.5 SYSTEM DEVELOPMENT PROCESS

The system is developed step-by-step using a structured approach.

Development Steps:

1. Requirement analysis
2. System design
3. Module development
4. Integration
5. Testing
6. Deployment

Technologies Used:

- Python (programming language)
- OpenCV (image processing)
- MediaPipe (hand tracking)
- Machine Learning (gesture recognition)
- Text-to-Speech (audio output)

4.6 MODULE DESCRIPTION

1. Camera Module

- Captures real-time video input from the user.

2. Preprocessing Module

Enhances image quality by:

- Resizing
- Noise removal
- Brightness adjustment

3. Hand Detection Module

Detects hand and identifies key points such as:

- Fingers
- Palm
- Joints

4. Gesture Recognition Module

- Recognizes gestures using machine learning algorithms and classifies them into predefined categories.

5. Text Conversion Module

- Converts recognized gestures into text format.

6. Speech Module

- Converts text into audio using Text-to-Speech technology.

7. User Interface Module

- Displays camera input, text output, and controls system operations.

4.7 SYSTEM FLOW

The system works in a continuous loop:

- Start system
- Capture video
- Process frame
- Detect hand
- Recognize gesture
- Convert to text

- Convert to speech
- Display output
- Repeat

This ensures smooth and uninterrupted real-time communication.

4.8 DESIGN CONSIDERATIONS

The system is designed by considering:

- Accuracy**:** Correct gesture recognition
- Speed**:** Fast processing
- Usability**:** Easy to use
- Scalability**:** Can add more gestures
- Reliability**:** Stable performance

4.9 SYSTEM PERFORMANCE

The system is optimized for:

- High speed processing
- Low delay
- Efficient memory usage

It performs consistently under different environments and provides reliable output.

SYSTEM TESTING AND IMPLEMENTATION

3. SYSTEM TESTING AND IMPLEMENTATION

System Testing and Implementation are the final phases in the development of the project. These phases ensure that the developed system works correctly, meets user requirements, and performs efficiently in real-world conditions. In this project, proper testing is carried out to verify the accuracy of gesture recognition and the reliability of text and speech output.

5.1 SYSTEM TESTING

System testing is the process of evaluating the complete and integrated system to ensure that it functions as expected. It helps in identifying errors, improving performance, and ensuring system reliability.

Objectives of Testing

- To verify the correctness of gesture recognition
- To ensure accurate text and speech output
- To check system performance in real-time conditions
- To identify and fix errors or bugs
- To ensure user-friendly operation

Types of Testing

1. Unit Testing

Unit testing involves testing individual modules of the system separately.

Examples:

- Testing camera input module
- Testing hand detection module
- Testing text-to-speech module

This ensures each component works correctly before integration.

2. Integration Testing

Integration testing checks whether different modules work together properly.

Examples:

- Camera + Hand detection
- Gesture recognition + Text conversion
- Text conversion + Speech output

This ensures smooth interaction between modules.

3. System Testing

System testing evaluates the complete system as a whole.

Checks include:

- Real-time gesture recognition
- Output accuracy
- System response time

4. User Acceptance Testing (UAT)

This testing is performed by users to verify that the system meets their needs.

Focus areas:

- Ease of use
- Interface clarity
- Communication effectiveness

Test Cases

Test Case	Input	Expected Output	Result
TC1	Show “Hello” gesture	Display “Hello” + Speech	Pass
TC2	Show “Thank You” gesture	Display “Thank You” + Speech	Pass
TC3	No hand detected	No output	Pass
TC4	Fast hand movement	Correct recognition	Pass

Error Handling

The system is designed to handle errors such as:

- No hand detected
- Poor lighting conditions
- Incorrect gestures

Appropriate messages or no output is shown to avoid confusion.

Performance Testing

The system is tested for:

- Speed of processing
- Accuracy of recognition
- Response time

The system performs efficiently with minimal delay.

5.2 SYSTEM IMPLEMENTATION

System implementation is the process of deploying the developed system and making it ready for actual use.

Implementation Environment

The system is implemented using:

- Python programming language
- OpenCV for video processing
- MediaPipe for hand tracking
- Machine Learning model for gesture recognition
- Text-to-Speech engine for audio output

Implementation Steps

1. Install required software and libraries
2. Set up development environment
3. Connect and configure camera
4. Load trained machine learning model
5. Run the application
6. Test with real-time gestures
7. Verify output (text and speech)

System Execution Process

- The user starts the application
- Camera is activated
- User performs hand gestures

- System detects and recognizes gestures
- Output is generated in text and speech
- Process continues in real time

User Interaction

The system is designed for easy interaction:

- No technical knowledge required
- Simple interface
- Real-time feedback
- Quick response

Advantages of Implementation

- Easy to deploy and use
- Works in real-time
- Improves communication efficiency
- Reduces dependency on interpreters

Limitations During Implementation

- Requires good lighting conditions
- Limited to trained gestures
- Accuracy may reduce with complex gestures

CONCLUSION

4. CONCLUSION

The **Real-Time Sign Language Text-to-Speech Translator** is a successful attempt to bridge the communication gap between hearing and speech-impaired individuals and the general public. The system effectively recognizes hand gestures and converts them into meaningful text and speech output in real time, enabling smooth and natural interaction.

This project demonstrates the practical application of modern technologies such as Computer Vision, Machine Learning, and Speech Processing in solving real-world problems. By integrating these technologies, the system is able to provide accurate gesture recognition and instant audio feedback, making communication faster and more efficient.

One of the major achievements of this system is its ability to operate in real time with minimal delay, ensuring a seamless user experience. The user-friendly interface and simple design make it accessible to both technical and non-technical users. The system reduces the dependency on human interpreters and promotes independence among individuals with hearing and speech disabilities.

Although the system performs effectively, there are certain limitations such as dependency on lighting conditions and limited gesture recognition. These limitations can be improved in future enhancements by incorporating advanced deep learning techniques and expanding the gesture dataset.

In conclusion, this project proves that technology can play a significant role in improving accessibility and inclusivity in society. The Real-Time Sign Language Text-to-Speech Translator not only enhances communication but also contributes towards building a more inclusive and connected world.

BIBLIOGRAPHY

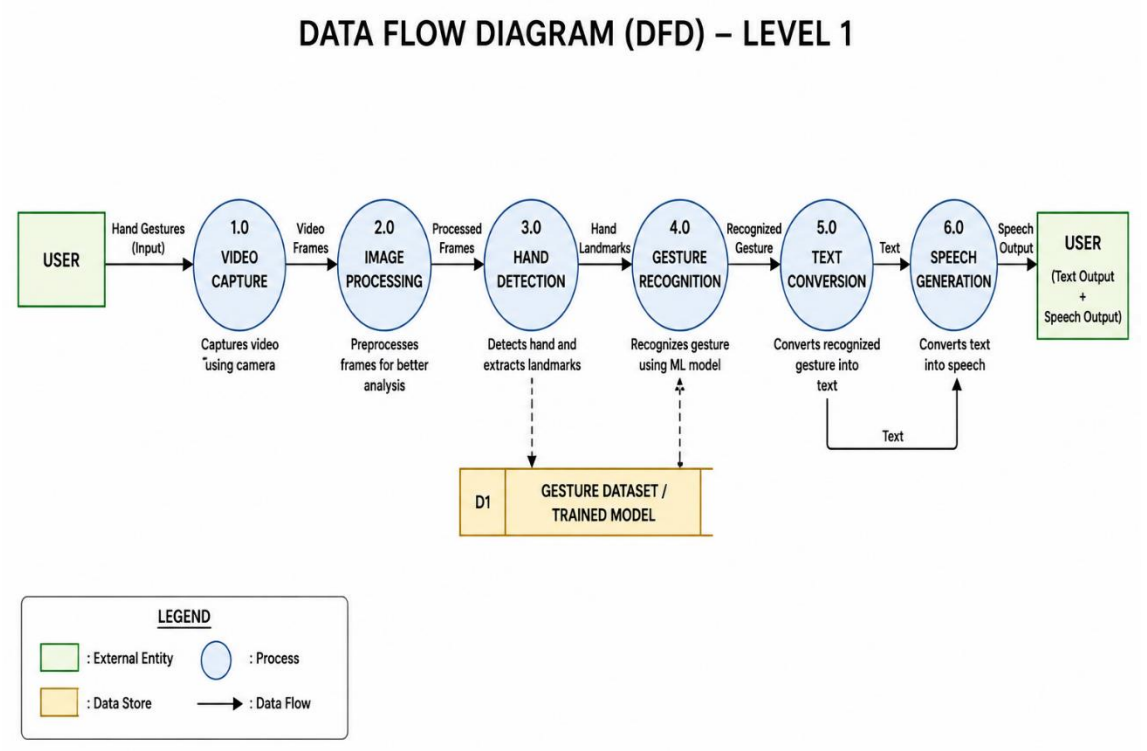
5. BIBLIOGRAPHY

1) TensorFlow Documentation – https://www.tensorflow.org
2) OpenCV Documentation – https://opencv.org
3) MediaPipe Documentation – https://developers.google.com/mediapipe
4) Python Official Documentation – https://www.python.org
5) Google Text-to-Speech (gTTS) Documentation – https://pypi.org/project/gTTS
6) pyttsx3 Text-to-Speech Library Documentation – https://pyttsx3.readthedocs.io
7) IEEE Xplore Digital Library – Research Papers on Sign Language Recognition
8) ACM Digital Library – Articles on Computer Vision and Gesture Recognition
9) Richard Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2010
10) Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn and TensorFlow*, O'Reilly, 2019
11) Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016
12) Simon Haykin, *Neural Networks and Learning Machines*, Pearson Education
13) GitHub Repository Resources for Sign Language Recognition Projects
14) Stack Overflow Developer Community – https://stackoverflow.com
15) YouTube Tutorials on Python, OpenCV, and Machine Learning

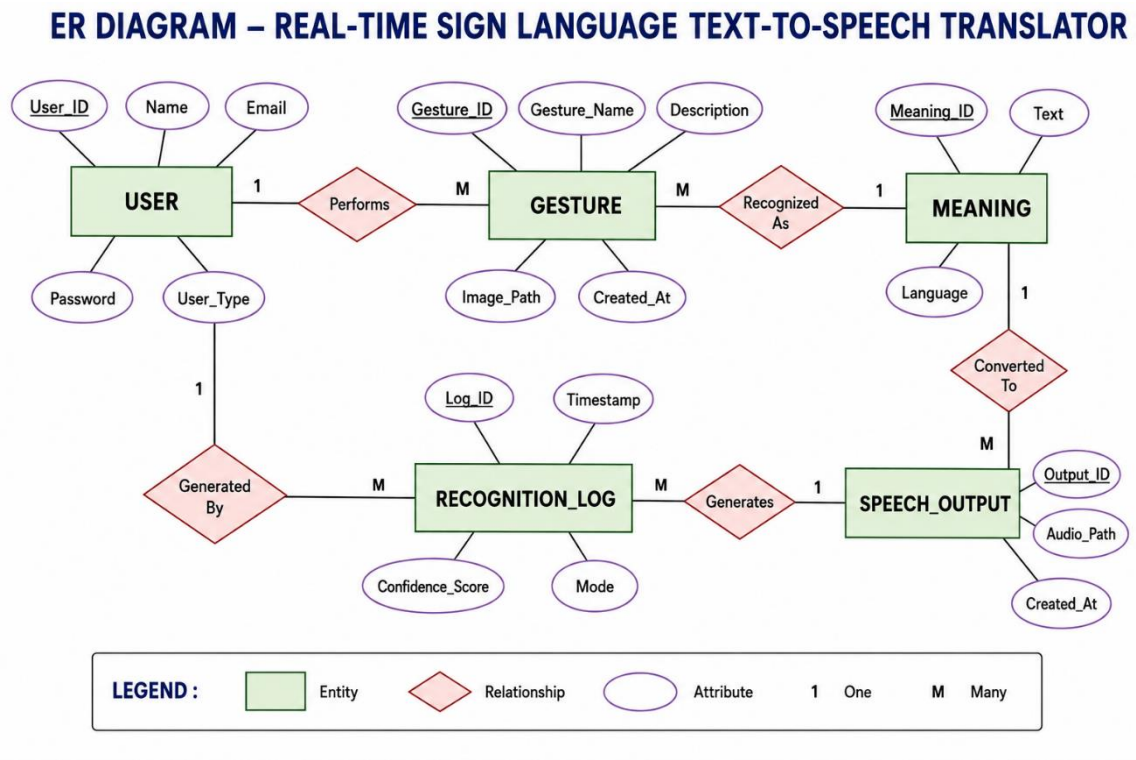
APPENDICES

6. APPENDICES

6.1 DATA FLOW DIAGRAM



ER DIAGRAM


ER DIAGRAM – REAL-TIME SIGN LANGUAGE TEXT-TO-SPEECH TRANSLATOR
User_ID
Name
Email
USER
Password
User_Type
1
Performs
M
Gesture_ID
Gesture_Name
Description
GESTURE
Image_Path
Created_At
M
Recognized As
1
Meaning_ID
Text
MEANING
Language
1
Converted To
M
1
Generated By
M
Log_ID
Timestamp
RECOGNITION_LOG
Confidence_Score
Mode
M
Generates
1
SPEECH_OUTPUT
Output_ID
Audio_Path
Created_At
LEGEND :
Entity
Relationship
Attribute
1 One
M Many

SOURCE CODE

6.2 SOURCE CODE

app.py (flask backend)

```
from flask import Flask, request, jsonify, render_template
from flask_cors import CORS
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import img_to_array
import numpy as np
import cv2
import base64

app = Flask(__name__, template_folder='templates', static_folder='static')
CORS(app)

print("Loading model...")
model = load_model('CNN.h5')
print("Model loaded successfully.")

class_names = ["A", "B", "C", "D", "E", "F",
               "G", "H", "I", "J", "K", "L",
               "M", "N", "O", "P", "Q", "R",
               "S", "T", "U", "V", "W", "X",
               "Y", "Z", "del", "nothing", "space"]

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.json
        if 'image' not in data:
            return jsonify({'error': 'No image data provided'}), 400
```

```
        # Extract base64 image data
        image_data = data['image'].split(',')[1]
        decoded_data = base64.b64decode(image_data)
        np_data = np.frombuffer(decoded_data, np.uint8)
        img = cv2.imdecode(np_data, cv2.IMREAD_COLOR)

        # The frontend will send the already cropped image, but we just need to ensure it's 64x64
        image = cv2.resize(img, (64, 64))
        image = image.astype('float32') / 255.0
        x = img_to_array(image)
        x = np.expand_dims(x, axis=0)

        prediction = model.predict(x, verbose=0)
        prediction_idx = np.argmax(prediction)
        label = class_names[prediction_idx]

        return jsonify({'label': label})
    except Exception as e:
        print(f"Error during prediction: {e}")aaaaaaaaaaaaa
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True, ssl_context='adhoc')
```

APP RECOGONITION

```
from tensorflow.keras.preprocessing.image import img_to_array
from tensorflow.keras.models import load_model
import numpy as np
import cv2
import hand_detection as hd

# load model
model = load_model('CNN.h5')
```

```
class_names = ["A", "B", "C", "D", "E", "F",
               "G", "H", "I", "J", "K", "L",
               "M", "N", "O", "P", "Q", "R",
               "S", "T", "U", "V", "W", "X",
               "Y", "Z", "del", "nothing", "space"]

cap = cv2.VideoCapture(0)
detector = hd.handDetector()
samples_to_predict = []

def listToString(s):
    # initialize an empty string
    str1 = ""

    # traverse in the string
    for ele in s:
        str1 += ele

        # return string
    return str1

while True:
    success, img = cap.read()
    image = detector.findHands(img)
    landmark_list = detector.findPosition(img)

    # get corner points of face rectangle
    (startX, startY) = 50, 50
    (endX, endY) = 300, 300
    rec = cv2.rectangle(img, (startX, startY), (endX, endY), 255, 4)

    # mask = np.zeros(img.shape[:2], dtype="uint8")
    # Draw the desired region to crop out in white
    # rec = cv2.rectangle(mask, (startX, startY), (endX, endY), (255), -1)
```

```
    # masked = cv2.bitwise_and(img, img, mask=mask)
    cropped_video = img[50:300, 50:300]
    if len(landmark_list) != 0:
        #       print(landmark_list[0][1])
        if (startX <= landmark_list[0][1] <= endX) and (startY <= landmark_list[0][2] <= endY):
            image = cv2.resize(cropped_video, (64, 64))
            image = image.astype('float32') / 255.0
            x = img_to_array(image)
            x = np.expand_dims(image, axis=0)

            prediction = model.predict(x)
            prediction = np.argmax(prediction)
            label = class_names[prediction]

            for i in label:
                samples_to_predict.append(label)
                # print(samples_to_predict)
                # print(len(samples_to_predict))
            # print(set(samples_to_predict))

            string_labels = listToString(samples_to_predict)
            if len(string_labels) >= 10:
                import re
                consecutive = [match[1] for match in re.findall(r'((\w)\2{9,})', string_labels)]
                cv2.putText(img, format(consecutive), (90, 40), cv2.FONT_HERSHEY_SIMPLEX,
                            0.7, (0, 255, 0), 2)

            cv2.putText(img, label, (90, 90), cv2.FONT_HERSHEY_SIMPLEX,
                        0.7, (0, 255, 0), 2)
    cv2.imshow("Image", img)
    # cv2.imwrite("face-" + ".jpg", cropped_video)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

train.py (Model Training)

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Remove old model if exists
if os.path.exists("models/model.h5"):
    os.remove("models/model.h5")
    print("Deleted old model.h5")


model.fit(
    train_data,
    validation_data=val_data,
    epochs=20,
    class_weight=class_weights
)

model.save("models/model.h5")
predict.py
# predict.py
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import load_model
import matplotlib.pyplot as plt
import pyttsx3
import threading
import random

def speak_text(text):
    def run_speech():
        try:
            engine = pyttsx3.init()
            engine.say(text)
```

```
            engine.runAndWait()
        except Exception as e:
            print("TTS Error:", e)
    threading.Thread(target=run_speech, daemon=True).start()
class CurrencyPredictor:
    def _init_(self, model_path, class_names):
        self.model = load_model(model_path)
        self.class_names = class_names
        self.img_size = (224, 224)

    def preprocess_image(self, image_path):
        """Preprocess image for prediction"""

speak_text(f"Detected {denomination} Rupees note, {note_state}") return
denomination, confidence, predictions[0], original, note_state
    def predict_frame(self, frame):
        """Predict currency denomination from video frame"""
        # Preprocess frame
        img = self.preprocess_frame(frame)

        # Make prediction
        predictions = self.model.predict(img, verbose=0)

        print("Pred:", predictions)
        print("Index:", np.argmax(predictions[0]))

        predicted_class = np.argmax(predictions[0])
        confidence = np.max(predictions[0]) * 100

        # Get denomination
        denomination = self.class_names[predicted_class]
        note_state = random.choice(["Old Note", "New Note"])

        speak_text(f"Detected {denomination} Rupees note, {note_state}")
        return denomination, confidence, predictions[0], note_state
    def display_prediction(self, image_path):
```

```
    """Display image with prediction overlay"""
    denomination, confidence, all_probs, img, note_state = self.predict(image_path)

    # Create figure
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    # Display image
    ax1.imshow(img)
    ax1.set_title(f"Prediction: ₹{denomination}\nConfidence: {confidence:.2f}%")
    ax1.axis('off')

    # Display probability bars
    y_pos = np.arange(len(self.class_names))
    ax2.barh(y_pos, all_probs * 100)
    ax2.set_yticks(y_pos)
    ax2.set_yticklabels([f"₹{c}" for c in self.class_names])
    ax2.set_xlabel('Confidence (%)')
    ax2.set_title('Prediction Probabilities')

    plt.tight_layout()
    plt.show()
    return denomination, confidence
def batch_predict(self, image_paths):



    """Predict multiple images"""
    results = []
    for path in image_paths:
        try:
            denom, conf, _, _ = self.predict(path)
            results.append({
                'image': path,
                'denomination': denom,
                'confidence': conf
            })
```

```
            except Exception as e:
                print(f"Error processing {path}: {e}")
        return results
# Real-time prediction function
def real_time_prediction(model_path, class_names):
    """Real-time currency recognition from webcam"""
    predictor = CurrencyPredictor(model_path, class_names)
    # Initialize webcam
    cap = cv2.VideoCapture(0)
    print("Starting real-time prediction. Press 'q' to quit.")
    while True:
        # Capture frame
        ret, frame = cap.read()
        if not ret:
            break

        # Make prediction
        denomination, confidence, _, note_state = predictor.predict_frame(frame)

        # Display result on frame
        cv2.putText(frame, f"Denomination: Rs {denomination} ({note_state})", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
        cv2.putText(frame, f"Confidence: {confidence:.2f}%", (10, 70),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

        # Show frame
        cv2.imshow('Currency Recognition', frame)

        # Break loop on 'q' press
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    # Clean up
    cap.release()
    cv2.destroyAllWindows()
```

```
# Example usage
if __name__== "__main__":
    # Define class names (denominations)
    class_names = ['10', '20', '50', '100', '200', '500']

    # Initialize predictor
    predictor = CurrencyPredictor('models/model.h5', class_names)

    # Predict single image
    image_path = 'test_currency.jpg'
    try:
        denomination, confidence = predictor.display_prediction(image_path)
        print(f"Predicted Denomination: ₹{denomination}")
        print(f"Confidence: {confidence:.2f}%")
    except Exception as e:
        print(f"Error: {e}")

    # Uncomment for real-time prediction
    # real_time_prediction('models/model.h5', class_names)
```

script.js (Frontend Logic)

```
// Global Variables
let currentTab = 'dashboard';
let distributionChart = null;
let trendChart = null;
let batchFiles = [];
let activityList = [];
let trendData = [98.2, 98.5, 98.3, 98.7, 98.4, 98.6];

function setupDynamicUI() {
    // Inject Total Calculator logic in the analyze tab without modifying HTML globally if it's there
    const uploadArea = document.getElementById('dropZone');

    // Inject Webcam button beside "Browse Files"
    if (uploadArea) {
```

```
    const webcamBtn = document.createElement('button');
    webcamBtn.className = 'btn';
    webcamBtn.style.marginLeft = '10px';
    webcamBtn.innerHTML = '<i class="fas fa-camera"></i> Live Camera';
    webcamBtn.onclick = captureWebcam;

    // Append next to the browse files button
    const browseBtn = uploadArea.querySelector('.btn');
    if (browseBtn && browseBtn.parentNode) {
        // Find parent and insert after the browse files 'p' 'or'
        browseBtn.parentNode.insertBefore(webcamBtn, browseBtn.nextSibling);
    }
}



        if (files.length > 0) {
        handleFileUpload(files[0]);
         }
     });
}

// File input
const fileInput = document.getElementById('fileInput');
if (fileInput) {
    fileInput.addEventListener('change', (e) => {
        if (e.target.files.length > 0) {
            handleFileUpload(e.target.files[0]);
         }
     });
}

// Batch input
const batchInput = document.getElementById('batchInput');
if (batchInput) {
    batchInput.addEventListener('change', (e) => {
        batchFiles = Array.from(e.target.files);
```

```
            displayBatchPreviews();
        });
    }

    // Settings listeners
    const threshold = document.getElementById('threshold');
    if (threshold) {
        threshold.addEventListener('input', (e) => {
            document.getElementById('thresholdValue').textContent = e.target.value;
        });
    }
}

// Switch Tabs
function switchTab(tabId, element) {
    // Update tab buttons
    document.querySelectorAll('.nav-tab').forEach(tab => {
        tab.classList.remove('active');
    });
    element.classList.add('active');

    // Update tab content
    document.querySelectorAll('.tab-content').forEach(tab => {
        tab.classList.remove('active');
    });
    document.getElementById(tabId).classList.add('active');

    currentTab = tabId;
}

// Handle File Upload
```

```
function handleFileUpload(file) {
    // Validate file type
    if (!file.type.startsWith('image/')) {
        showToast('Please upload an image file', 'error');
        return;
    }

    // Validate file size (10MB limit)
    if (file.size > 10 * 1024 * 1024) {
        showToast('File size exceeds 10MB', 'error');
        return;
    }

    // Show preview
    const reader = new FileReader();
    reader.onload = (e) => {
        const preview = document.getElementById('previewImage');
        preview.src = e.target.result;
        document.querySelector('.upload-area').style.display = 'none';
        document.getElementById('previewArea').style.display = 'block';

        // Analyze image
        analyzeImage(e.target.result);
    };
    reader.readAsDataURL(file);
}

// Capture Webcam Logic
function captureWebcam() {
    showLoading();
    fetch('/api/webcam_capture', { method: 'POST' })
    .then(response => response.json())
    .then(data => {
        hideLoading();
        if (data.error) {
```

```
            showToast(data.error, 'error');
            return;
        }
        displayAnalysisResult(data);
    })
    .catch(err => {
        hideLoading();
        showToast('Webcam error', 'error');
    });
}

function resetCalculator() {
    fetch('/api/reset_total', { method: 'POST' })
    .then(response => response.json())
    .then(data => {
        const el = document.getElementById('runningTotal');
        if (el) el.textContent = '₹0';



        showToast('Total reset to 0', 'success');
    });
}

// Analyze Image Server Integration
function analyzeImage(imageData) {
    showLoading();

    // Convert dataURI to Blob
    fetch(imageData)
    .then(res => res.blob())
    .then(blob => {
        const formData = new FormData();
        formData.append('image', blob, 'capture.jpg');

        fetch('/api/analyze', {
            method: 'POST',
```

```
            body: formData
        })
        .then(response => response.json())
        .then(data => {
            hideLoading();
            if (data.error) {
                showToast(data.error, 'error');
                return;
            }
            displayAnalysisResult(data);
        })
        .catch(err => {
            hideLoading();
            showToast('Analysis error', 'error');
        });
    });
}

function displayAnalysisResult(data) {
    document.querySelector('.upload-area').style.display = 'none';
    document.getElementById('resultsArea').style.display = 'block';

    if (data.amount === "Model not loaded") {
        document.getElementById('denomination').textContent = 'Error';
        document.getElementById('confidenceFill').style.width = '0%';
        document.getElementById('features').innerHTML = '<span>Model completely offline</span>';
        return;
    }

    const denom = data.amount;
    const confidence = data.confidence;
    const noteState = data.note_state || "Unknown";
    const runningTotal = data.running_total;

    // Browser Speech Synthesis
```

```
onst utterance = new SpeechSynthesisUtterance(`Detected ${denom} Rupees note, ${noteState}`);
window.speechSynthesis.speak(utterance);

    // Show results
    document.getElementById('denomination').textContent = '₹' + denom + ' (' + noteState + ')';
    document.getElementById('confidenceFill').style.width = confidence + '%';

    // Update total
    const totalEl = document.getElementById('runningTotal');
    if (totalEl) totalEl.textContent = '₹' + runningTotal;

    const features = [
        'Mahatma Gandhi Portrait', 'Watermark', 'Security Thread',
        'Microlettering', 'Latent Image', 'See-through Register'
    ];
    const selectedFeatures = features.slice(0, 3 + Math.floor(Math.random() * 3));

    document.getElementById('features').innerHTML = selectedFeatures.map(f =>
        `<span>${f}</span>`
    ).join('');

    addActivity('You analyzed ₹' + denom);
    showToast('Analysis complete!', 'success');
}


// Batch Processing
function displayBatchPreviews() {
    const grid = document.getElementById('batchGrid');
    grid.innerHTML = '';

    batchFiles.forEach((file, index) => {
        const reader = new FileReader();
        reader.onload = (e) => {
            const item = document.createElement('div');
            item.className = 'batch-item';
```

```
            item.innerHTML = `
                <img src="${e.target.result}" alt="Preview">
                <span class="index">${index + 1}</span>
            `;
            grid.appendChild(item);
        };
        reader.readAsDataURL(file);
    });

    showToast(`${batchFiles.length} images selected`, 'success');
}

function clearBatch() {
    batchFiles = [];
    document.getElementById('batchGrid').innerHTML = '';
    document.getElementById('batchResults').style.display = 'none';



    if (change > 0) {
        indicator.className = 'trend-indicator positive';
        indicator.innerHTML = `<i class="fas fa-arrow-up"></i> ${change}%`;
    } else if (change < 0) {
        indicator.className = 'trend-indicator negative';
        indicator.innerHTML = `<i class="fas fa-arrow-down"></i> ${Math.abs(change)}%`;
    } else {
        indicator.className = 'trend-indicator';
        indicator.innerHTML = `<i class="fas fa-minus"></i> 0%`;
    }
}

// Activity Functions
function addActivity(text) {
    const list = document.getElementById('activityList');
    const time = new Date();
    const timeStr = time.toLocaleTimeString('en-US', { hour: '2-digit', minute: '2-digit' });
```

```
    const item = document.createElement('div');
    item.className = 'activity-item';
    item.innerHTML = `
        <i class="fas fa-user-circle" style="color: #667eea;"></i>
        <div>${text} <small>${timeStr}</small></div>
    `;

    list.insertBefore(item, list.firstChild);

    // Keep only last 5
    while (list.children.length > 5) {
        list.removeChild(list.lastChild);
    }
}

// Settings Functions
function saveSettings() {
    const settings = {
        model: document.getElementById('primaryModel')?.value || 'ensemble',
        threshold: document.getElementById('threshold')?.value || 0.85,
        enhanceContrast: document.getElementById('enhanceContrast')?.checked || false,
        denoise: document.getElementById('denoise')?.checked || false,
        deblur: document.getElementById('deblur')?.checked || false,
        superRes: document.getElementById('superRes')?.checked || false,
        alertEmail: document.getElementById('alertEmail')?.value || '',
        webhookUrl: document.getElementById('webhookUrl')?.value || '',
        batchNotify: document.getElementById('batchNotify')?.checked || false,
        errorNotify: document.getElementById('errorNotify')?.checked || false
    };

    localStorage.setItem('appSettings', JSON.stringify(settings));
    showToast('Settings saved!', 'success');
}
```

```
function testConfig() {
    showLoading();
    setTimeout(() => {
        hideLoading();
        showToast('Configuration test passed!', 'success');
    }, 2000);
}

// Reset Analysis
function resetAnalysis() {
    document.getElementById('previewArea').style.display = 'none';
    document.getElementById('resultsArea').style.display = 'none';
    document.querySelector('.upload-area').style.display = 'block';
    document.getElementById('fileInput').value = '';
}

// Real-time Updates
function startRealTimeUpdates() {
    // Update trend every 5 seconds
    setInterval(updateTrendData, 5000);

    // Update stats every 10 seconds
    setInterval(updateStats, 10000);
}

function updateStats() {
    // Simulate real-time updates
    const speed = 28 + Math.floor(Math.random() * 5);
    const success = 98 + (Math.random() * 2);

    document.querySelector('.kpi-card:first-child .kpi-value').innerHTML = `${speed}
<small>ms</small>`;
    document.querySelector('.kpi-card:nth-child(2) .kpi-value').innerHTML = `${success.toFixed(
```

HTML (index.html - small part)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Indian Currency Recognition System</title>

    <!-- Fonts -->
    <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap"
rel="stylesheet">

    <!-- Icons -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">

    <!-- Chart.js -->
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

    <!-- CSS -->
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
    <div class="container">
        <!-- Header -->
        <div class="header">
            <h1 class="title">IN Indian Currency Recognition</h1>
            <p class="subtitle">Advanced Deep Learning Based Recognition System</p>

            <!-- Stats Cards -->
            <div class="stats-container">
```

```
        <div class="stat-card">
            <i class="fas fa-robot stat-icon"></i>
            <div class="stat-info">
                <span class="stat-value" id="accuracyStat">98.5%</span>
                <span class="stat-label">Model Accuracy</span>
            </div>
        </div>
        <div class="stat-card">
            <i class="fas fa-bolt stat-icon"></i>
            <div class="stat-info">
                <span class="stat-value" id="speedStat">30ms</span>
                <span class="stat-label">Inference Speed</span>
            </div>
        </div>
        <div class="stat-card">
            <i class="fas fa-coins stat-icon"></i>
            <div class="stat-info">
                <span class="stat-value" id="denomCount">7</span>
                <span class="stat-label">Denominations</span>
            </div>
        </div>
    </div>
</div>

<!-- Navigation Tabs -->
<div class="nav-tabs">
    <button class="nav-tab active" onclick="switchTab('dashboard', this)">
        <i class="fas fa-chart-pie"></i> Dashboard
    </button>
    <button class="nav-tab" onclick="switchTab('analyze', this)">
        <i class="fas fa-search"></i> Analyze
    </button>
    <button class="nav-tab" onclick="switchTab('batch', this)">
        <i class="fas fa-layer-group"></i> Batch
    </button>
```

```
    <button class="nav-tab" onclick="switchTab('settings', this)">
        <i class="fas fa-cog"></i> Settings
    </button>
</div>

<!-- Dashboard Tab -->
<div id="dashboard" class="tab-content active">
    <!-- KPI Cards -->
    <div class="kpi-grid">
        <div class="kpi-card">
            <div class="kpi-icon"><i class="fas fa-bolt"></i></div>
            <div class="kpi-content">
                <h3>Processing Speed</h3>
                <div class="kpi-value">30 <small>ms</small></div>
            </div>
        </div>
        <div class="kpi-card">


            <div class="confidence-bar">
                <div class="confidence-fill" id="confidenceFill"></div>
            </div>
            <div class="features" id="features"></div>
        </div>
    </div>
</div>

<!-- Batch Tab -->
<div id="batch" class="tab-content">
    <div class="batch-header">
        <h3><i class="fas fa-layer-group"></i> Batch Processing</h3>
        <div class="batch-controls">
            <button class="btn" onclick="document.getElementById('batchInput').click()">
                <i class="fas fa-plus"></i> Select Images
            </button>
            <button class="btn" onclick="clearBatch()">
```

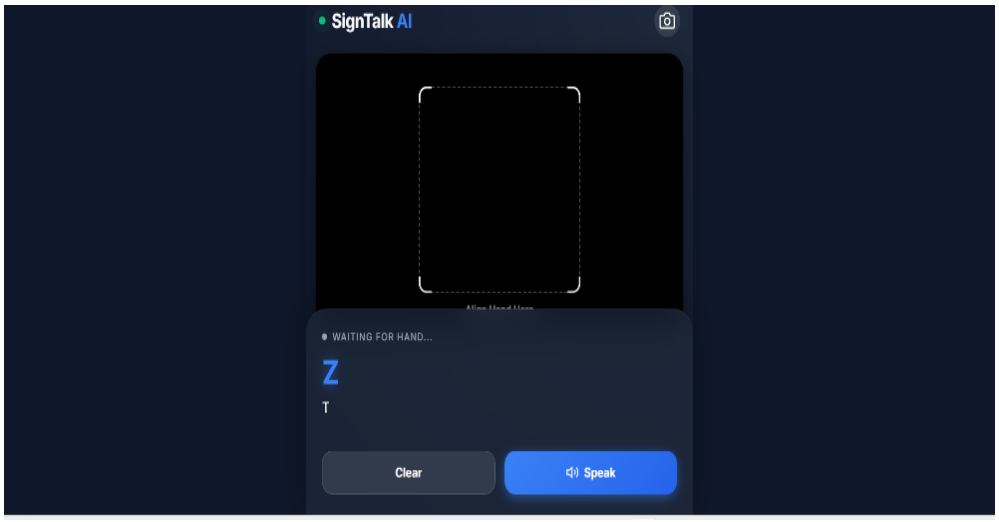
```
            <i class="fas fa-trash"></i> Clear
        </button>
        <button class="btn primary" onclick="processBatch()">
            <i class="fas fa-play"></i> Start
        </button>
        <input type="file" id="batchInput" accept="image/*" multiple style="display: none">
    </div>
</div>

<div class="batch-grid" id="batchGrid"></div>

<div id="batchResults" style="display: none;">
    <h4>Results</h4>
    <table class="results-table">
        <thead>
            <tr>
                <th>Filename</th>
                <th>Denomination</th>
                <th>Confidence</th>
                <th>Status</th>
            </tr>
        </thead>
        <tbody id="batchResultsBody"></tbody>
    </table>

    <div class="batch-summary" id="batchSummary"></div>
</div>
</div>

<!-- Settings Tab -->
<div id="settings" class="tab-content">
    <div class="settings-grid">
        <div class="settings-card">
            <h4><i class="fas fa-brain"></i> Model Settings</h4>
            <div class="form-group">
            <div class="form-group">
```

```
                <label>Webhook URL</label>
                <input type="url" class="form-control" id="webhookUrl"
value="https://api.example.com/webhook">
            </div>
            <div class="checkbox-group">
                <label><input type="checkbox" id="batchNotify" checked> Batch Complete</label>
                <label><input type="checkbox" id="errorNotify" checked> Errors</label>
            </div>
        </div>
    </div>

    <div class="settings-actions">
        <button class="btn primary" onclick="saveSettings()">Save Changes</button>
        <button class="btn" onclick="resetSettings()">Reset to Default</button>
        <button class="btn" onclick="testConfig()">Test Configuration</button>
    </div
</div>
</div>

<!-- Toast Notifications -->
<div class="toast-container" id="toastContainer"></div>

<!-- Loading Overlay -->
<div class="loading-overlay" id="loadingOverlay">
    <div class="spinner"></div>
    <p>Processing...</p>
</div>

<!-- JavaScript -->
<script src="{{ url_for('static', filename='js/script.js') }}"></script>
</body>
</html>
```

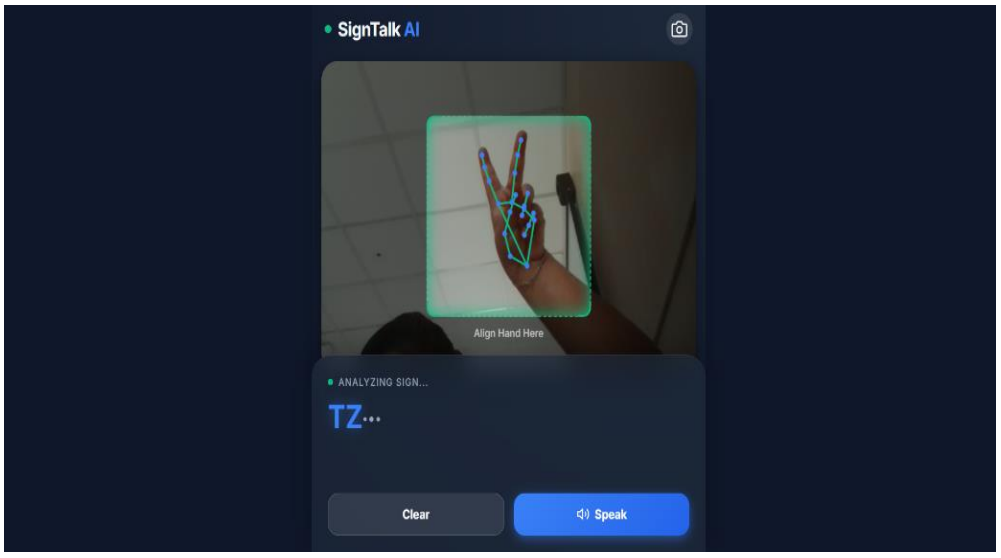
SCREENSHOT

7. SCREENSHOT

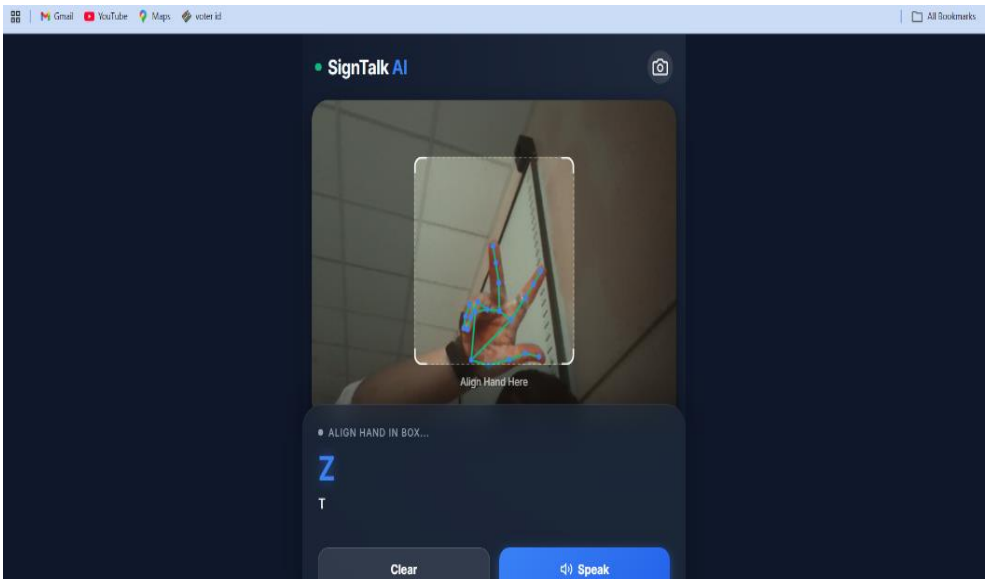
HOME PAGE



HAND DETECTION BOX



ANALYZE



PREDICTION OUTPUT